Xinle (Eric) Song

310-869-9560 | ericss311@ucla.edu | linkedin.com/in/xinle-song | github.com/EricSongXinLe

Education

University of California, Los Angeles — Los Angeles, CA
Bachelor of Science in Computer Engineering — *Expected June 2027*

- GPA: 3.93/4.0; Dean's Honors List; IEEE-HKN Board Officer; recipient of three engineering scholarships
- Coursework: Algorithms and Complexity, Operating Systems, Parallel and Distributed Computing, Database Systems, Hardware for Machine Learning, Advanced Computer Architecture

Experience

Google — Jun. 2026 – Present
Technical Solutions Consultant Intern – AI Infrastructure

- Independently scoped and engineered an **end-to-end multimodal AI agent** with **LangGraph** that analyzes **10,000+ YouTube ads** using **Gemini 3.6 Flash** and generates structured analytical outputs.
- Implemented an **author-reviewer agent loop** that iterates analyses until reviewer approval, using **Protocol Buffers** to define structured model outputs and inter-agent messages.
- Cut **F1 distributed SQL** write time from **33 minutes to 18 seconds** (**110× faster**) by replacing per-record inserts with multi-row **UNION ALL** queries, reducing network round trips.
- Reduced unrecovered **F1 batch-write failures** from **∼5–6% to <1%** by implementing automatic retries with progressive backoff for rate-limited F1 API requests.
- Built **idempotent, restart-safe recovery** that reuses persisted results, reducing redundant Gemini traffic by **∼90–95%**.

Tencent — Aug. 2025 – Sep. 2025
Software Engineer Intern – AI Infrastructure

- Ported and deployed **∼10 proprietary 13B–70B LLMs** in a vLLM-based inference stack across **Hygon DCUs and NVIDIA GPUs**; validated model accuracy and benchmarked **TTFT, latency, and throughput** across accelerators.
- Reduced full-suite runtime from **8 hours to 30 minutes** (**16× faster**) by building a reusable **Python regression pipeline** for data ingestion, batch inference, output validation, benchmarking, and report generation.
- Extended the vLLM serving stack with **C++ license validation** exposed to Python through **pybind11** and encrypted-weight loading, enabling protected model initialization for enterprise deployments.

UCLA ORCAS Lab — Jan. 2025 – Present
Undergraduate Research Assistant – GPU Systems and Performance

- First-authored **CycleFence**, a low-overhead GPU profiling framework presented at **OSCAR 2026**, co-located with ISCA.
- Reduced profiling overhead by **61%** (49 to 19 cycles) by implementing warp-level synchronization and branch-free profiling APIs in the Vortex RISC-V GPGPU **C++** simulator.
- Designed and implemented **2:4 structured-sparsity** in **C++** for a thread-scalable SIMT Tensor Core, including weight compression, metadata decoding, and instruction scheduling, achieving up to **1.57× speedup** over dense execution.
- Built a GPU benchmark suite for **Llama 2 inference**, FlashAttention, and CNN workloads; automated configuration sweeps, cycle-level metric collection, and regression testing through **GitHub Actions CI**.

Projects

High-Performance GEMM Optimization | *C++, OpenMP, SIMD, perf, Linux* — Apr. 2025 – Jun. 2025

- Developed **OpenMP** and **MPI** implementations of **4096×4096 GEMM** across x86 and ARM systems.
- Optimized the OpenMP implementation using cache tiling, register blocking, multithreading, and SIMD vectorization to improve parallelism and memory locality.
- Implemented a separate MPI variant using row-wise data decomposition, **MPI_Scatterv**, **MPI_Bcast**, and **MPI_Gatherv** to distribute computation and collect rank-local results.
- Profiled cache misses and memory bottlenecks with Linux **perf**; achieved up to **633× speedup** over a single-threaded baseline, reaching **133 GFLOP/s** on an Intel Xeon CPU and **300 GFLOP/s** on Apple M1.

Pintos Operating System Kernel | *C, x86, System Calls, Process Management, Synchronization* — Nov. 2025 – Dec. 2025

- Extended the Pintos **x86 kernel** with user-process startup, command-line parsing, stack construction, and program loading.
- Implemented **13 system calls**, per-process file descriptor tables, and synchronized filesystem access using kernel locks.
- Built **exec/wait process coordination** with semaphores and reference-counted state, and validated user pointers through page-table mappings to prevent kernel faults.

Technical Skills

Languages: C++, Python, C, SQL, SystemVerilog, RISC-V Assembly
Technologies: LangGraph, vLLM, PyTorch, TensorRT, CUDA, OpenMP, MPI, SIMD, multithreading, Protobuf, pybind11
Tools: Linux/Unix, Git, GitHub Actions CI, Docker, Bazel, CMake, Make, GCC, GDB, Nsight Compute, perf, gem5, Codex